Programação Orientada a Objetos em Java

Prof. Msc Marcos de Souza

Design Patterns

FACULDADE IBRATEC

Objetivos

- Visam resolver problemas específicos
- Reuso de código
- Reduzem a complexidade

Origem

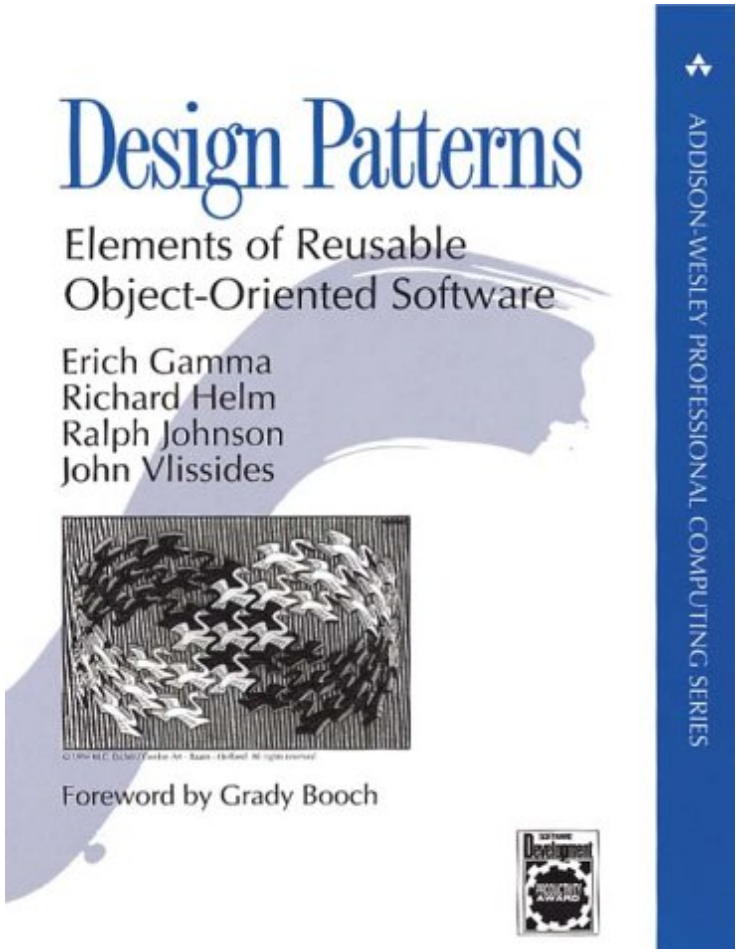
Em 1995, Erich Gamma, John Vlissides, Ralph Jonhson e Richard Helm descreveram 23 padrões que podem ser aplicados ao desenvolvimento de sistemas de software orientados a objetos.

Não são invenções. É uma documentação de soluções obtidas através da experiência.

Foram coletados de experiências de sucesso na indústria de software, principalmente de projetos em C++ e SmallTalk.

Origem

- Gang of Four (GoF)
  - ❑ E. Gamma and R. Helm and R. Johnson and J. Vlissides. Design Patterns - Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.



Definição

- Padrões de projeto estão relacionados a questões de comportamento de objetos, ciclo de vida de objetos, a interface dos objetos e a relacionamentos estruturais entre os objetos.

- Padrões de projeto utilizam eficientemente polimorfismo, herança, modularidade, composição, abstração para construir código reutilizável, eficiente, de alta coesão e baixo acoplamento.

Categorias

- Criacionais
- Estrutura
- Comportamento

Criacionais

- Tratam a inicialização de objetos;
- Reuso de código para diminuir a criação desnecessária de instâncias (objetos);
- Unificam regras para a configuração de objetos que são acessados de forma compartilhada.

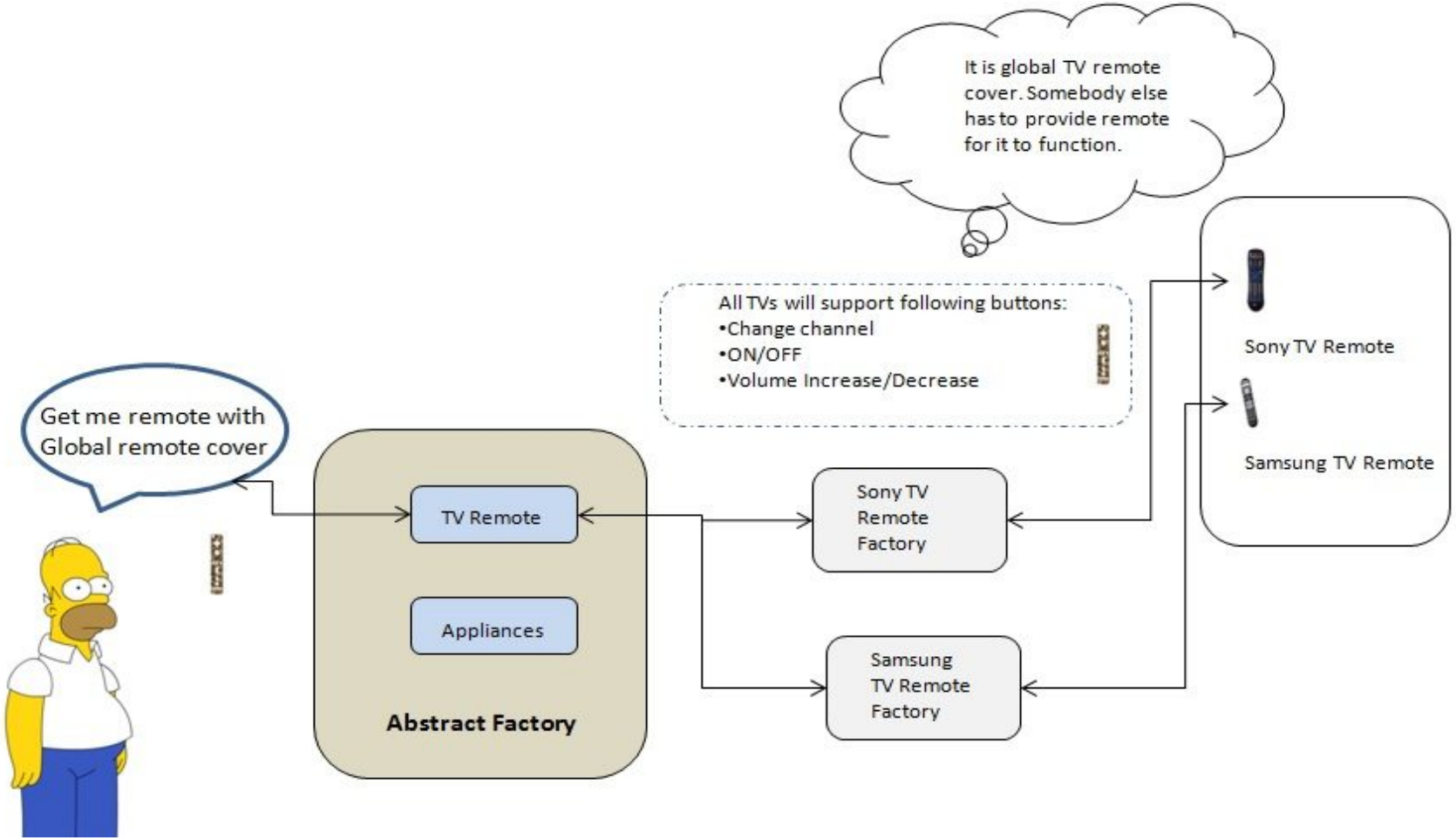
Criacionais

- Factory
- Abstract Factory
- Singleton

Factory

- Muitas vezes precisamos de um modelo onde determinados tipos de objetos precisem ser construídos de forma adaptável;
- Uma forma de encapsular a implementação de como esses objetos sejam criados é através de fábricas de objetos (factory);

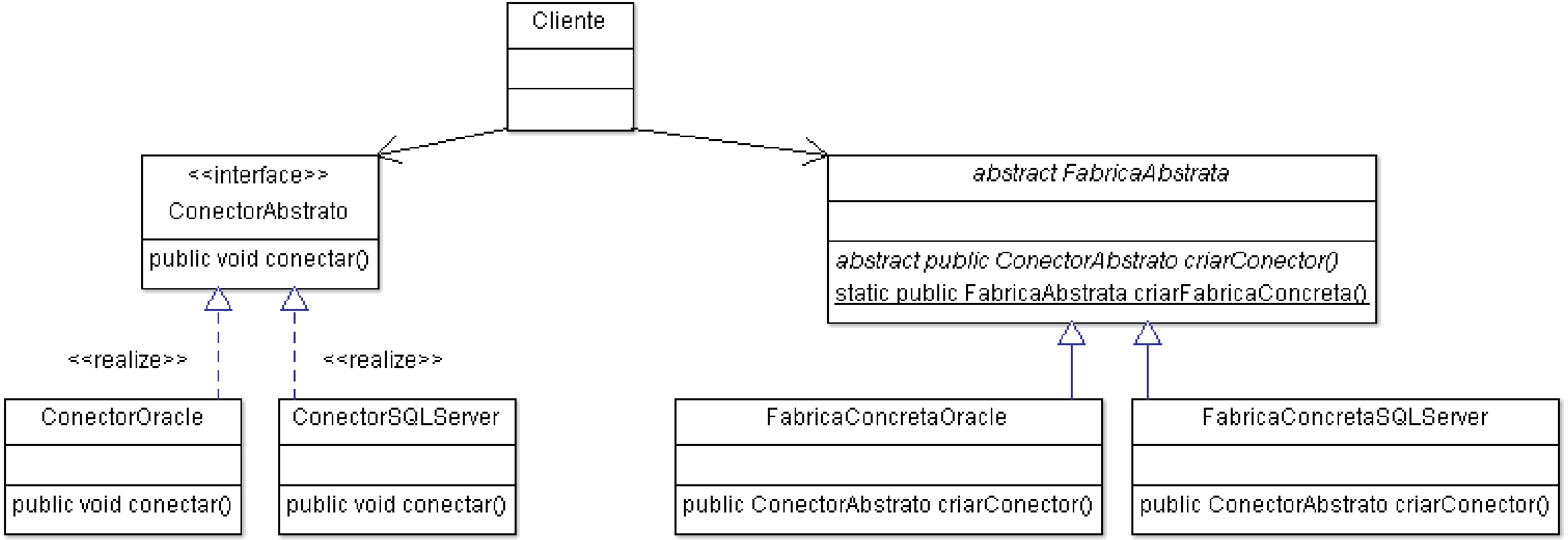
Factory


It is global TV remote cover. Somebody else has to provide remote for it to function.
All TVs will support following buttons:
•Change channel
•ON/OFF
•Volume Increase/Decrease
Sony TV Remote
Samsung TV Remote
Get me remote with Global remote cover
TV Remote
Appliances
Abstract Factory
Sony TV Remote Factory
Samsung TV Remote Factory

Abstract Factory

- Utilizado quando queremos ter um nível de abstração acima do padrão factory

Abstract Factory



Singleton

- Motivação: algumas classes devem ser instanciadas uma única vez:
  - Um *spooler* de impressão;
  - Um sistema de arquivos;
  - Um gerenciador de janelas;
  - Um objeto que contém a configuração do programa;
  - Um ponto de acesso ao banco de dados.

Singleton

- Problemas:
  - Programa precisa garantir que apenas uma instância de uma classe é criada.
  - Quando o objeto centraliza o gerenciamento de serviços:
    - Como criar uma instância que será global em Java?
    - Como fazer clientes capturar a mesma referência da instância única?

Singleton

```
public class Gerenciador {
  private static Gerenciador instancia;

  private Gerenciador() {
   ... //construcao do objeto
  }

  public static Gerenciador obterInstancia() {
    if (instancia == null) {
      instancia = new Gerenciador();
    }
    return instancia;
  }
```

Estruturais

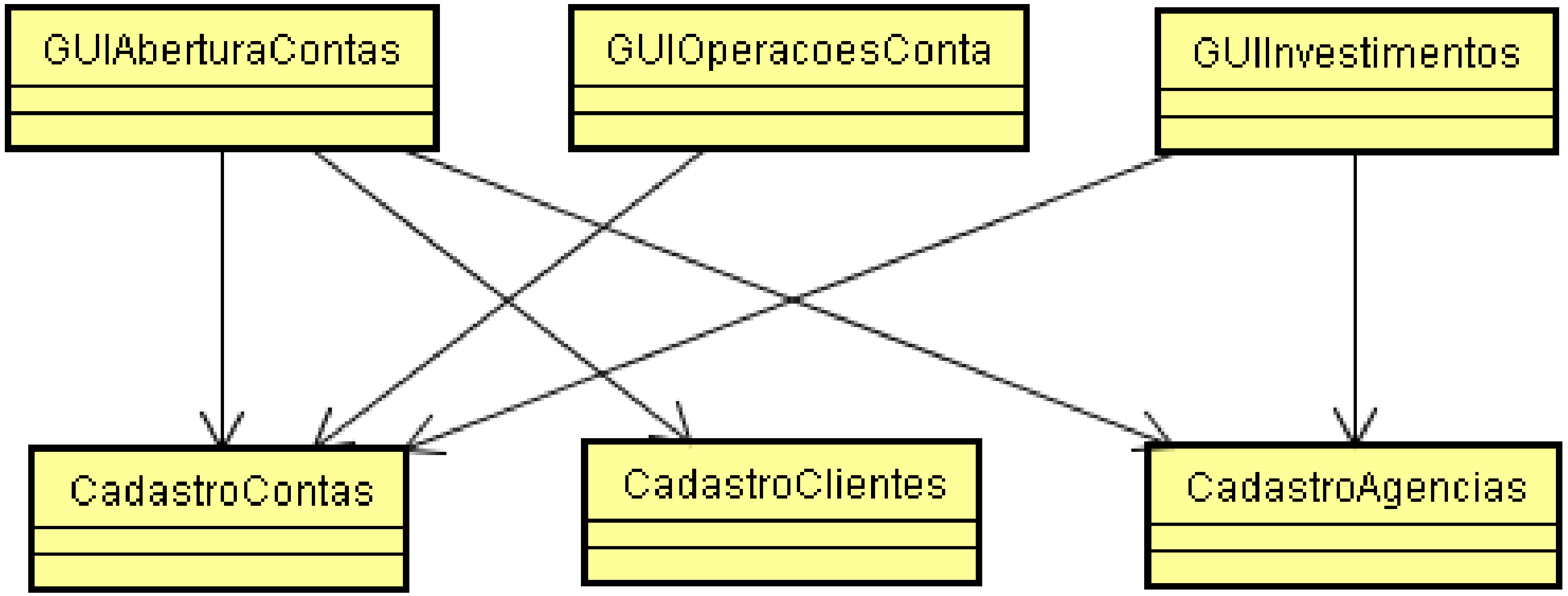
- Definem como será feita a composição entre as classes e seus objetos.

- Exemplos:
  - ***Adapter***
  - *Iterator*
  - ***Façade***
  - *Bridge*
  - *Decorator*

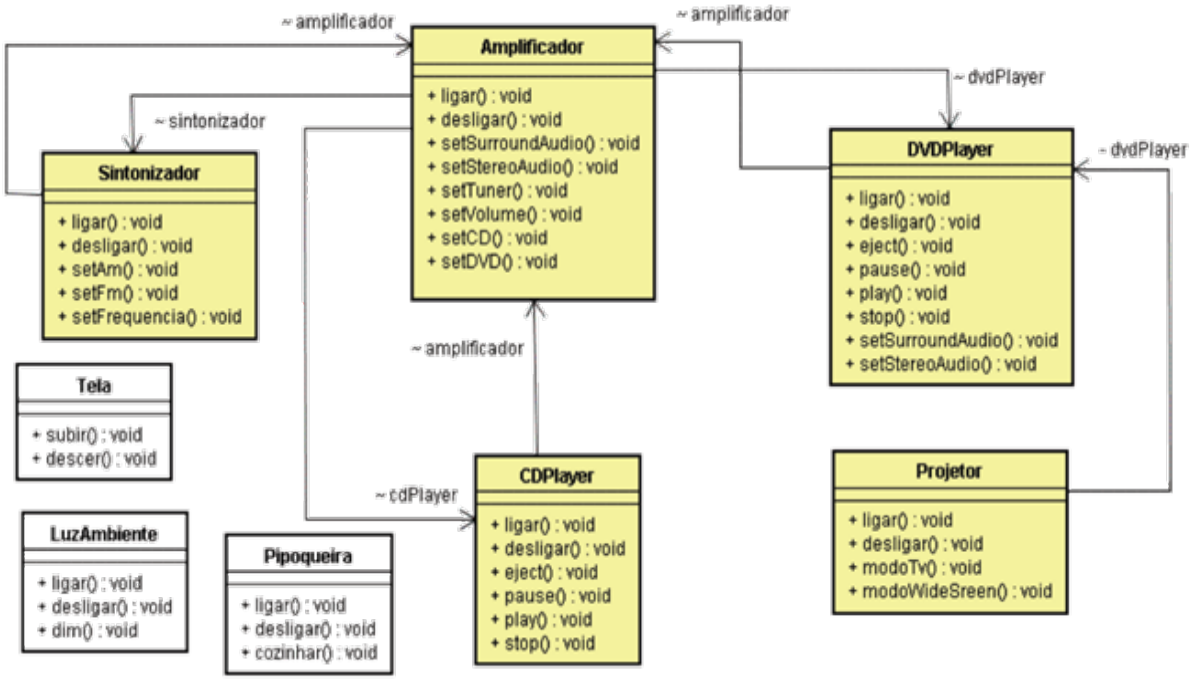
Fachada (Façade)

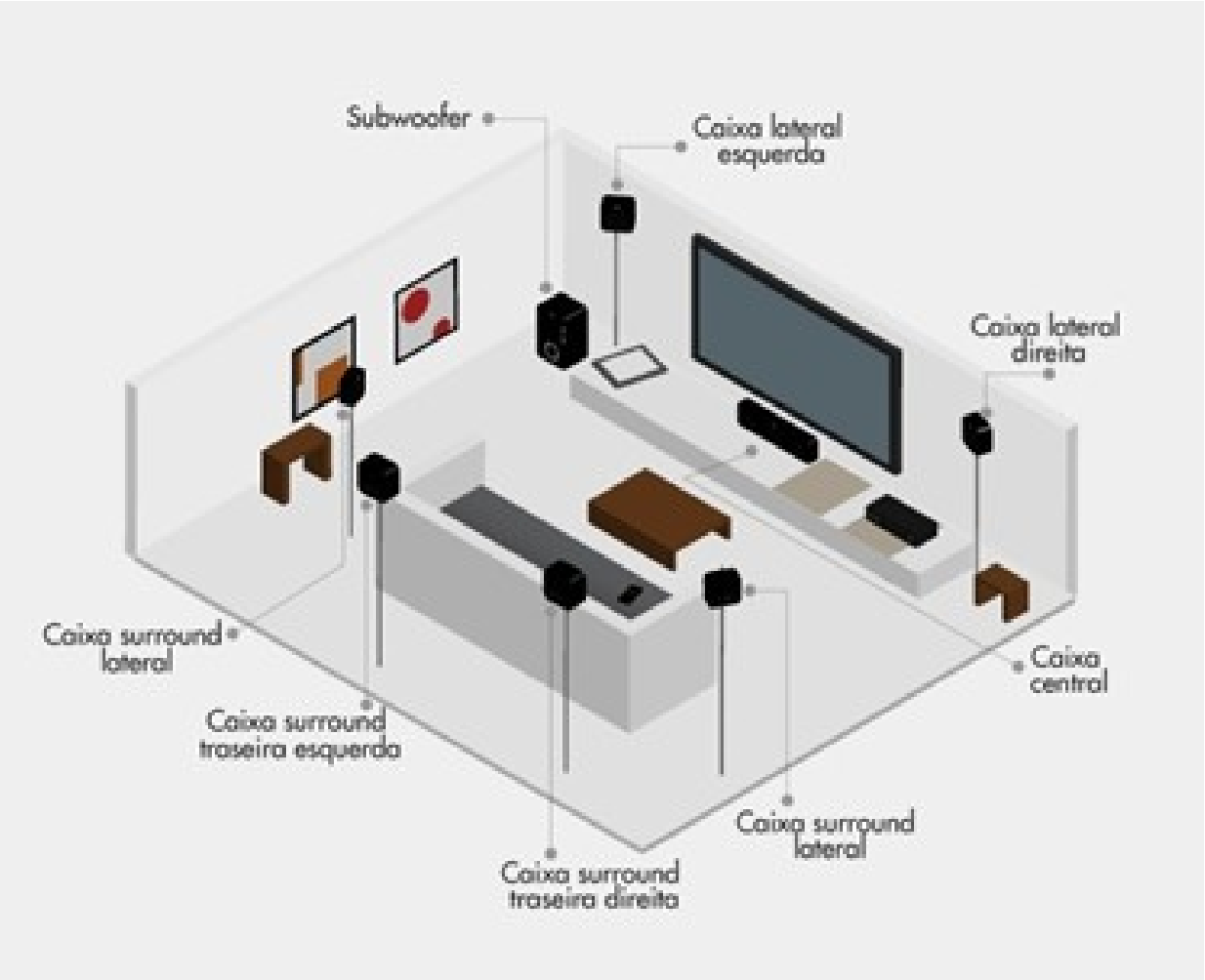
- Problemas:
  - Classes que necessitam de serviços de várias outras classes (“subsistemas”).
  - Precisamos fornecer uma forma padrão de acesso ao sistema.
    - Simplificar as classes que requisitam serviços
  - Exemplo: Classes de interface com o usuário pedindo serviços à parte de negócio de uma banco.
    - Subsistemas de contas, clientes, agências, etc...

Fachada (Façade)



Fachada (Façade)

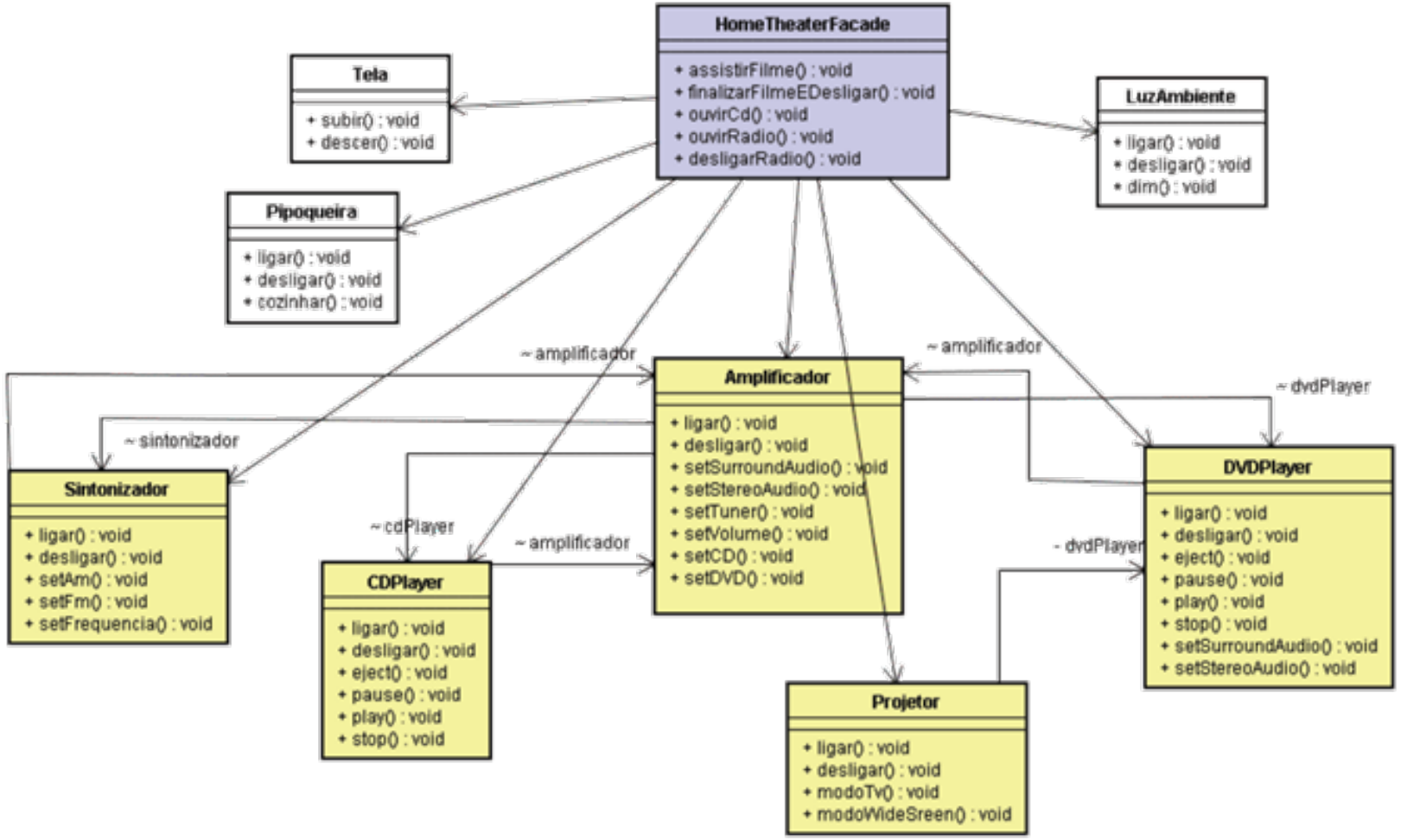




Fachada (Façade)

Como diminuir a complexidade do sistema, utilizando o encapsulamento de métodos?

Fachada (Façade)



Fachada (Façade)

```java
/**
   * Método que simplifica o uso de vários objetos
   * @param filme
   */
  public void assistirFilme(Filme filme) {
    System.out.println(
     "Tudo ok para assistir um filme!");
    pipoqueira.ligar();
    pipoqueira.cozinhar();

    luz.dim(10);

    tela.descer();

    projetor.ligar();
    projetor.setInput(dvdPlayer);
    projetor.modoWideSreen();

    amplificador.ligar();
    amplificador.setDVD(dvdPlayer);
    amplificador.setSurroundAudio();
    amplificador.setVolume(5);

    dvdPlayer.ligar();
    dvdPlayer.play(filme);
  }
```

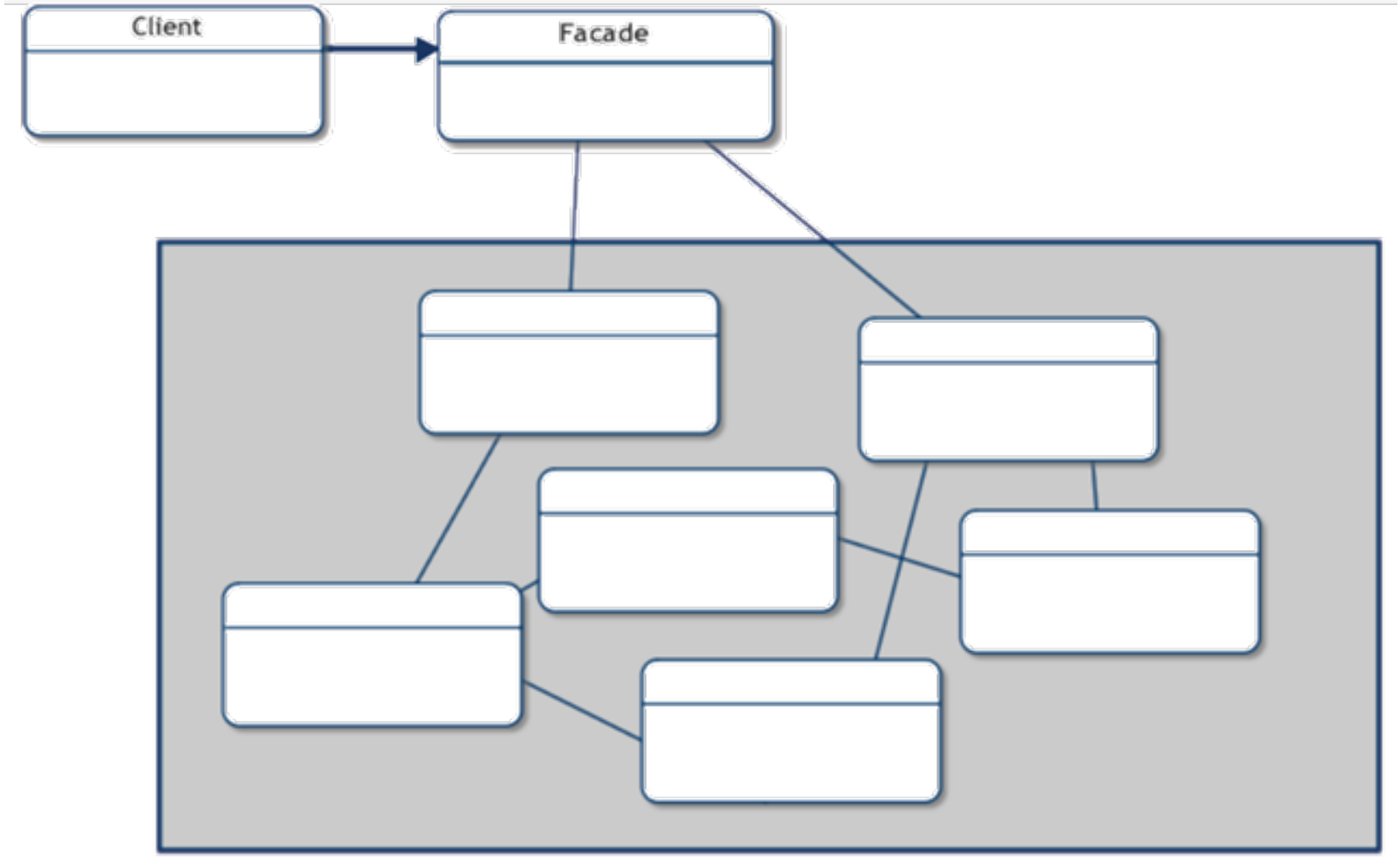
```java
/**
  * Outro método que simplifica
  * um conjunto de tarefas
  */
 public void pararFilme(){
   System.out.println("Desligando o
   home...");
   pipoqueira.desligar();
   luz.ligar();
   tela.subir();
   projetor.desligar();
   amplificador.desligar();
   dvdPlayer.parar();
   dvdPlayer.eject();
   dvdPlayer.desligar();
 }
```

Fachada (Façade)

- Solução:
  - Incluir uma classe Fachada intermediária
    - Interface única para os serviços do sistema;
    - Simplifica as classes;
    - Concentra a inteligência de acesso aos subsistemas em uma classe.
  - Define o protocolo de comunicação do sistema como um todo.

Fachada (Façade)

- Estrutura

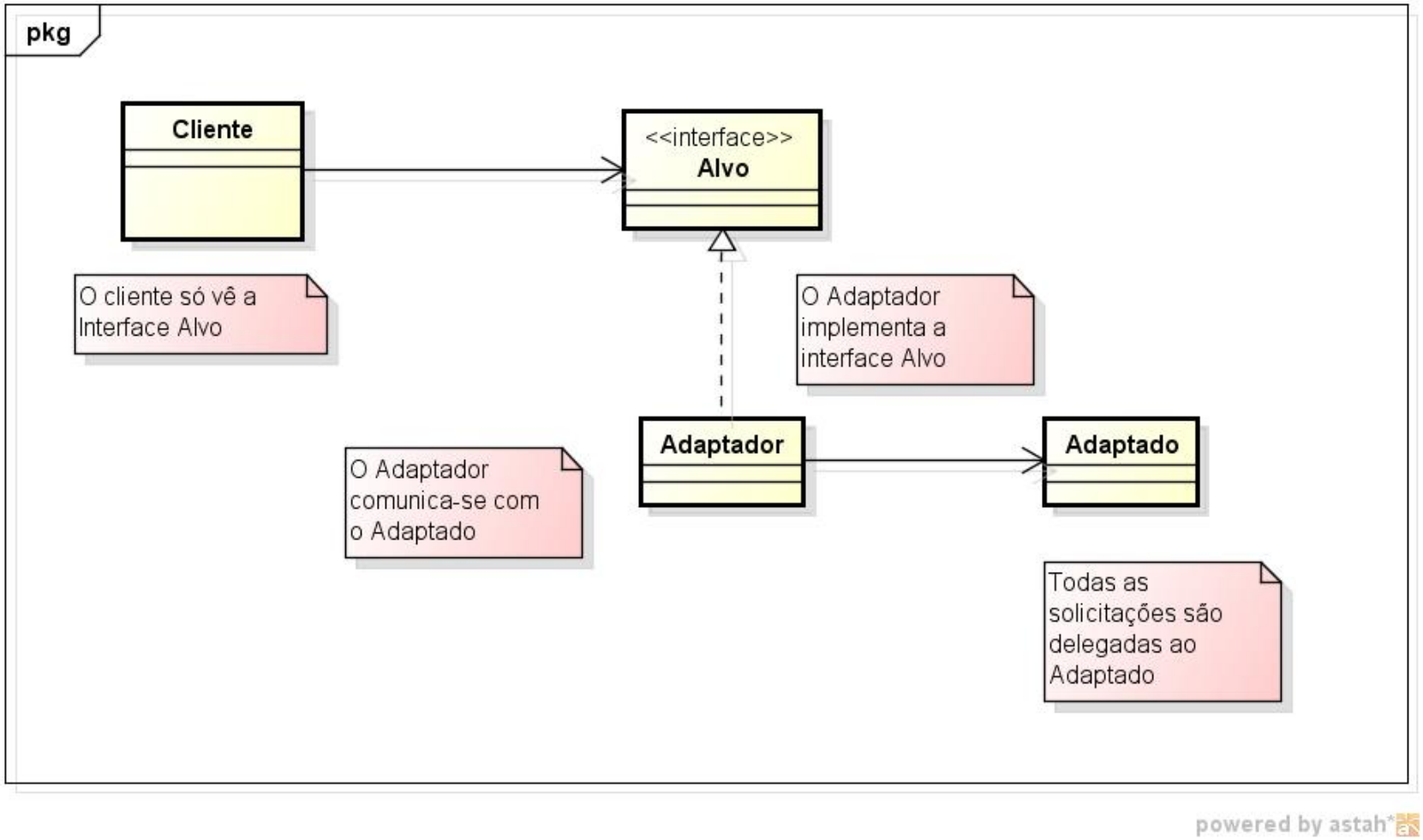


Adapter

- É comum em linguagens fortemente tipadas precisarem se **adaptar** a tipos diferentes de objetos

Adapter

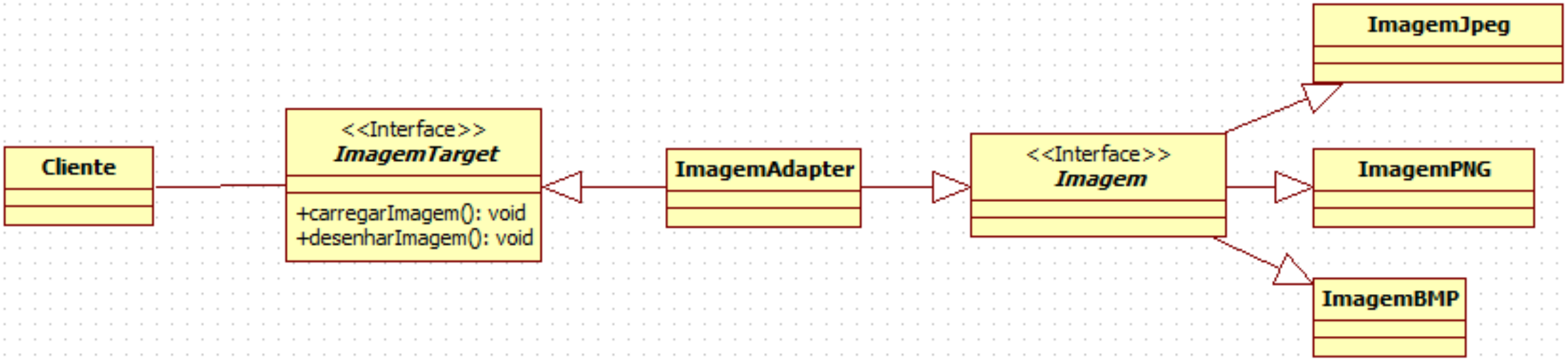
- Estrutura



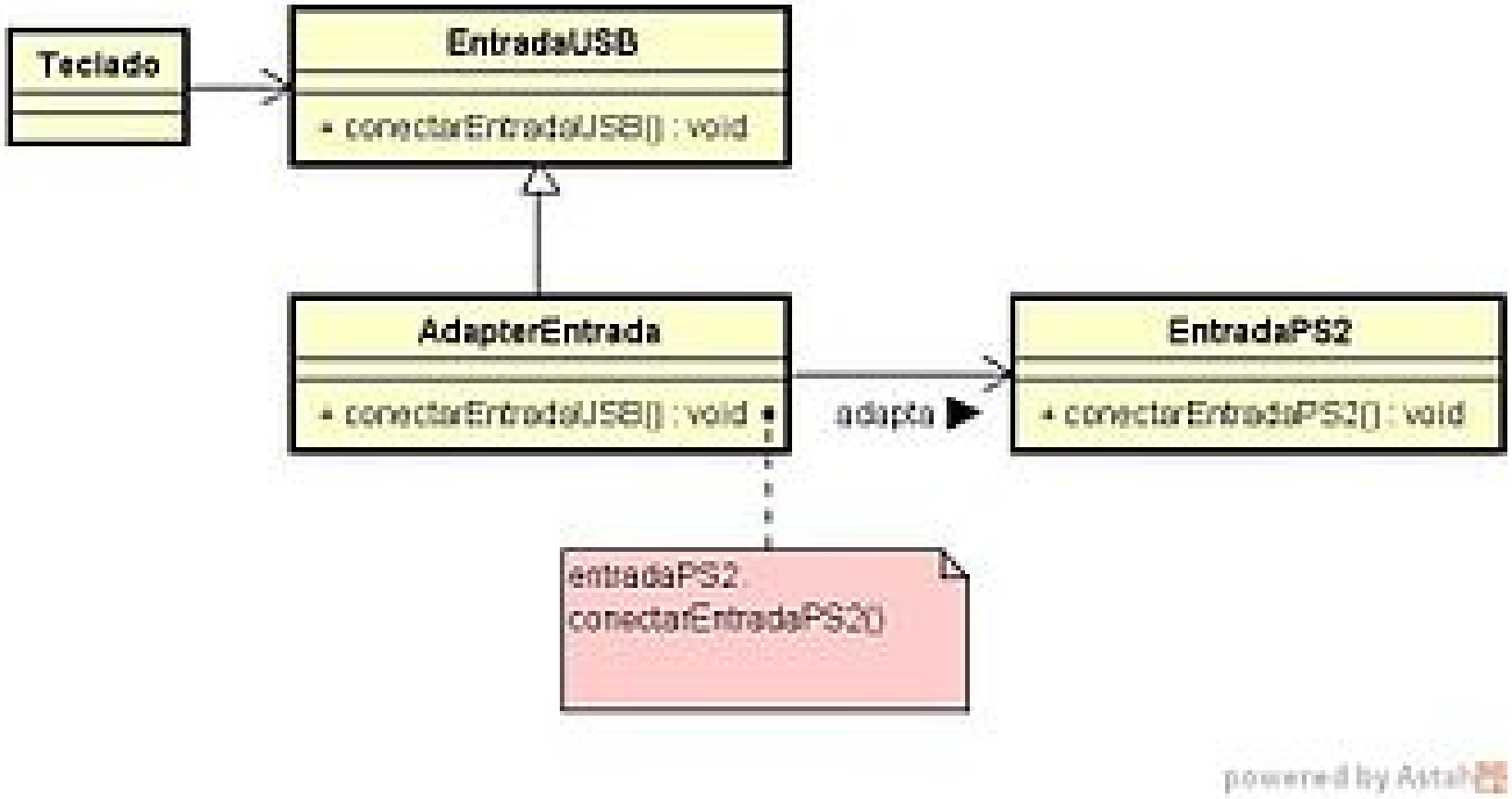
Adapter

- Exemplo Clássico

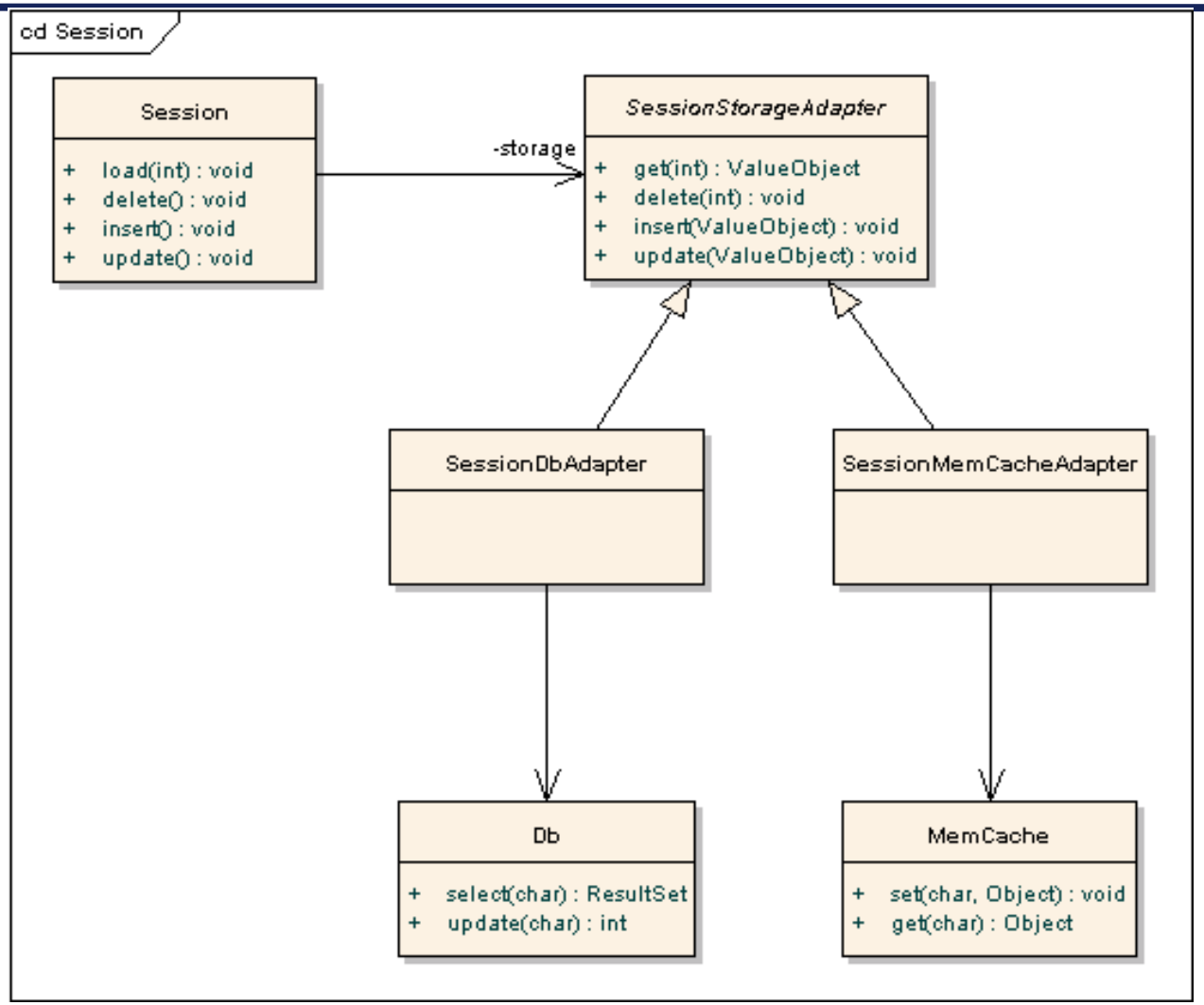
Adapter


Cliente
<<Interface>>
ImagemTarget
+carregarImagem(): void
+desenharImagem(): void
ImagemAdapter
<<Interface>>
Imagem
ImagemJpeg
ImagemPNG
ImagemBMP

Adapter


Teclado
EntradaUSB
+ conectarEntradaUSB() : void
AdapterEntrada
+ conectarEntradaUSB() : void
adapta
EntradaPS2
+ conectarEntradaPS2() : void
entradaPS2:
conectarEntradaPS2()
powered by Astah

Adapter

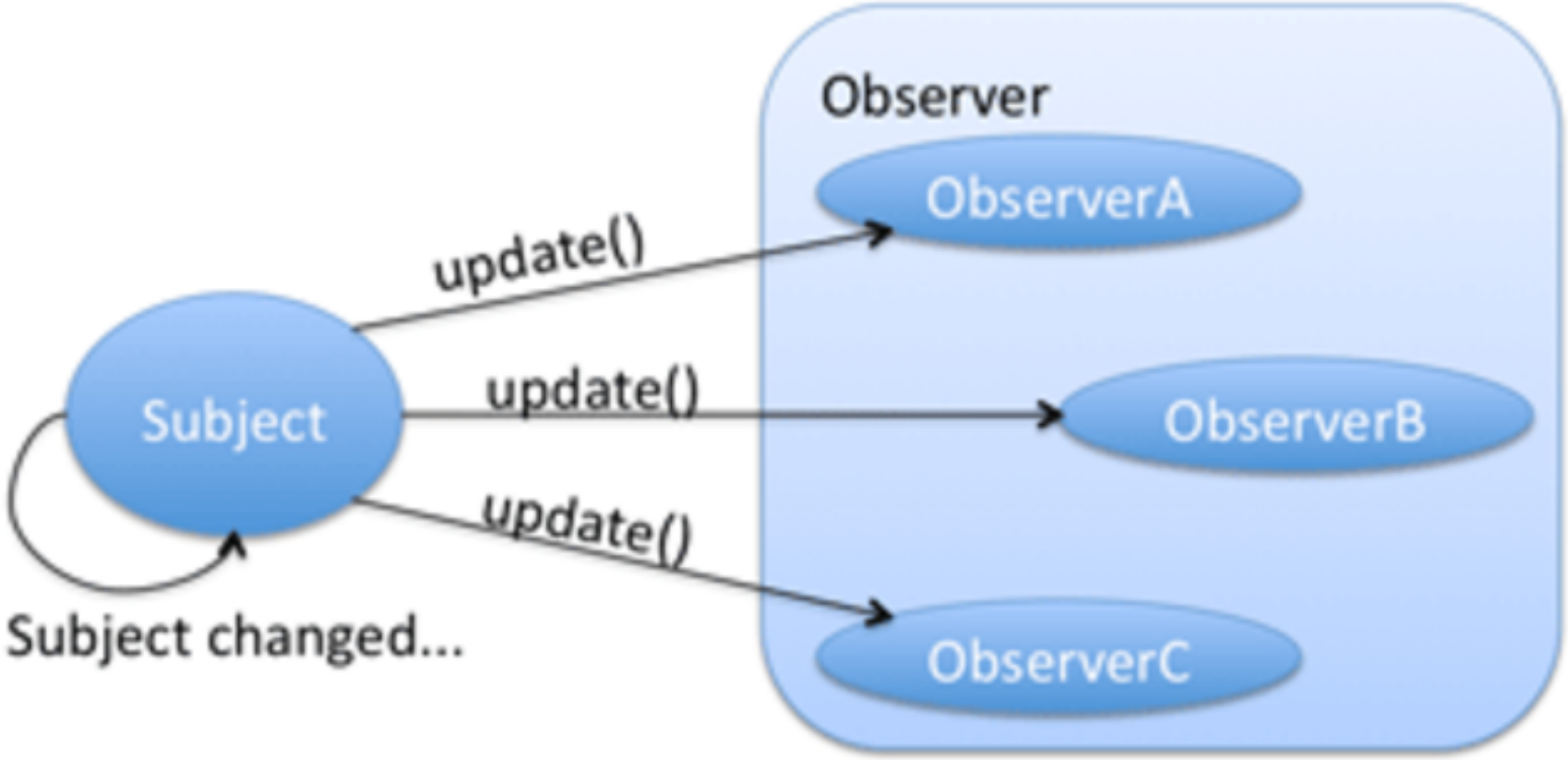

cd Session
Session
+ load(int) : void
+ delete() : void
+ insert() : void
+ update() : void
-storage
SessionStorageAdapter
+ get(int) : ValueObject
+ delete(int) : void
+ insert(ValueObject) : void
+ update(ValueObject) : void
SessionDbAdapter
SessionMemCacheAdapter
Db
+ select(char) : ResultSet
+ update(char) : int
MemCache
+ set(char, Object) : void
+ get(char) : Object

Comportamento

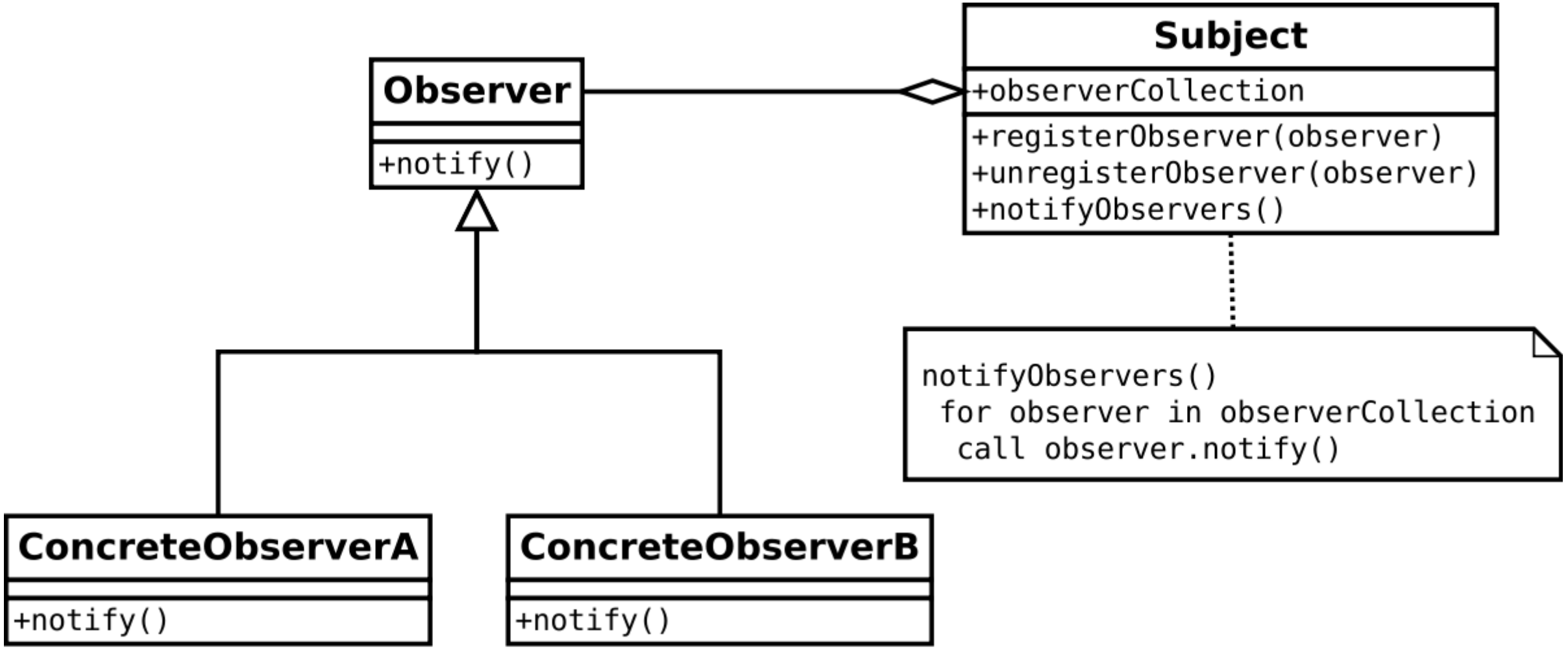
- Caracteriza a forma como os objetos irão interagir para realizar um dado comportamento.

  - ***Observer***
  - *Command*
  - *State*
  - *Strategy*
  - *Template Method*

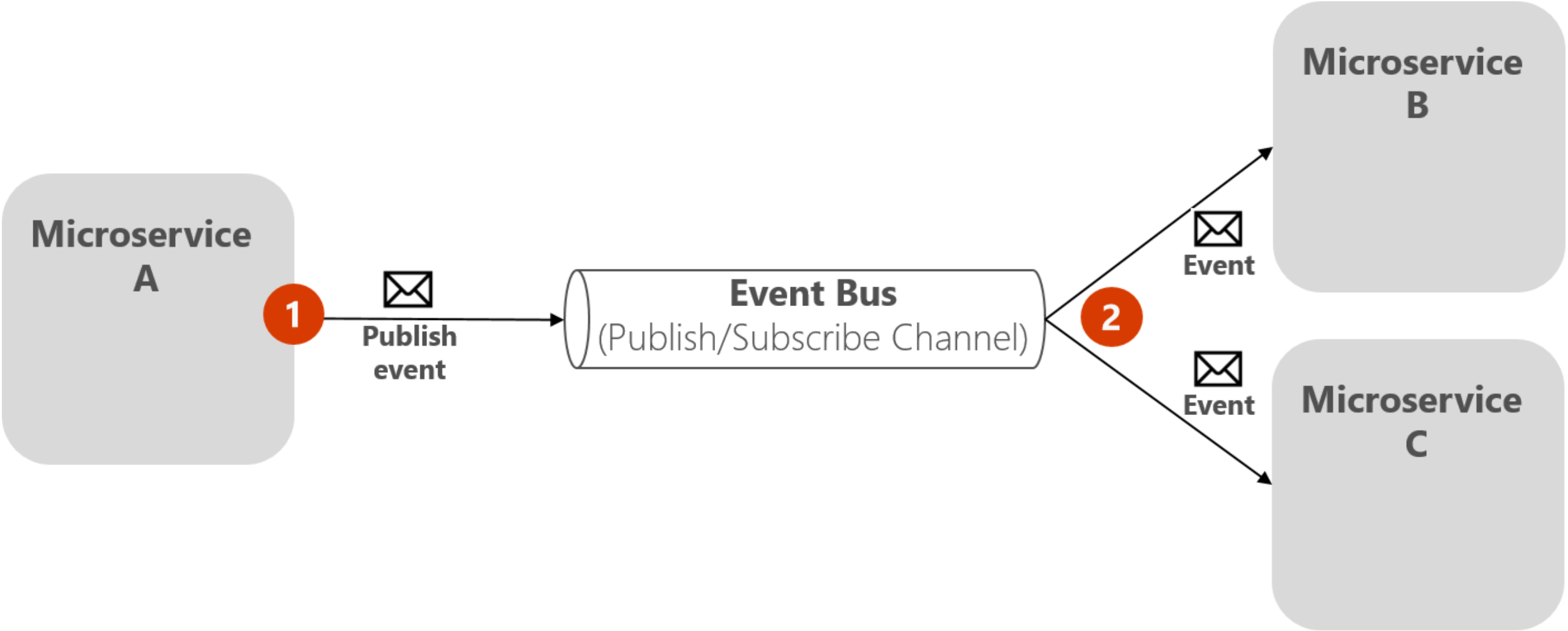
Observer


Observer
ObserverA
update()
Subject
update()
ObserverB
update()
Subject changed...
ObserverC

Observer


Subject
+observerCollection
+registerObserver(observer)
+unregisterObserver(observer)
+notifyObservers()
Observer
+notify()
notifyObservers()
for observer in observerCollection
call observer.notify()
ConcreteObserverA
+notify()
ConcreteObserverB
+notify()

Observer


Microservice A
1
Publish event
Event Bus
(Publish/Subscribe Channel)
2
Event
Event
Microservice B
Microservice C

Desenvolvimento em Camadas

FACULDADE IBRATEC

Objetivos

- Uma das grandes vantagens de se desenvolver em camadas é a possibilidade de separarmos componentes por responsabilidades em comum.

- Existem muitos programadores que acham desnecessário se preocupar com toda essa estruturação.

Objetivos

- Separa o código de acesso aos dados do código de negócio e interface com o usuário.
  - Modularidade

- Motivação
  - Facilidade para mudança de meios de persistência de dados (arquivo, BD).
  - Reusabilidade das camadas do sistema.

Objetivos

- Alguns sistemas cliente-servidor são constituídos de duas camadas:
  - Camada Cliente: encapsulam as camadas de apresentação e da lógica de negócios;
  - Camada Servidor: responsável pela persistência de dados (SGBD).

Objetivos

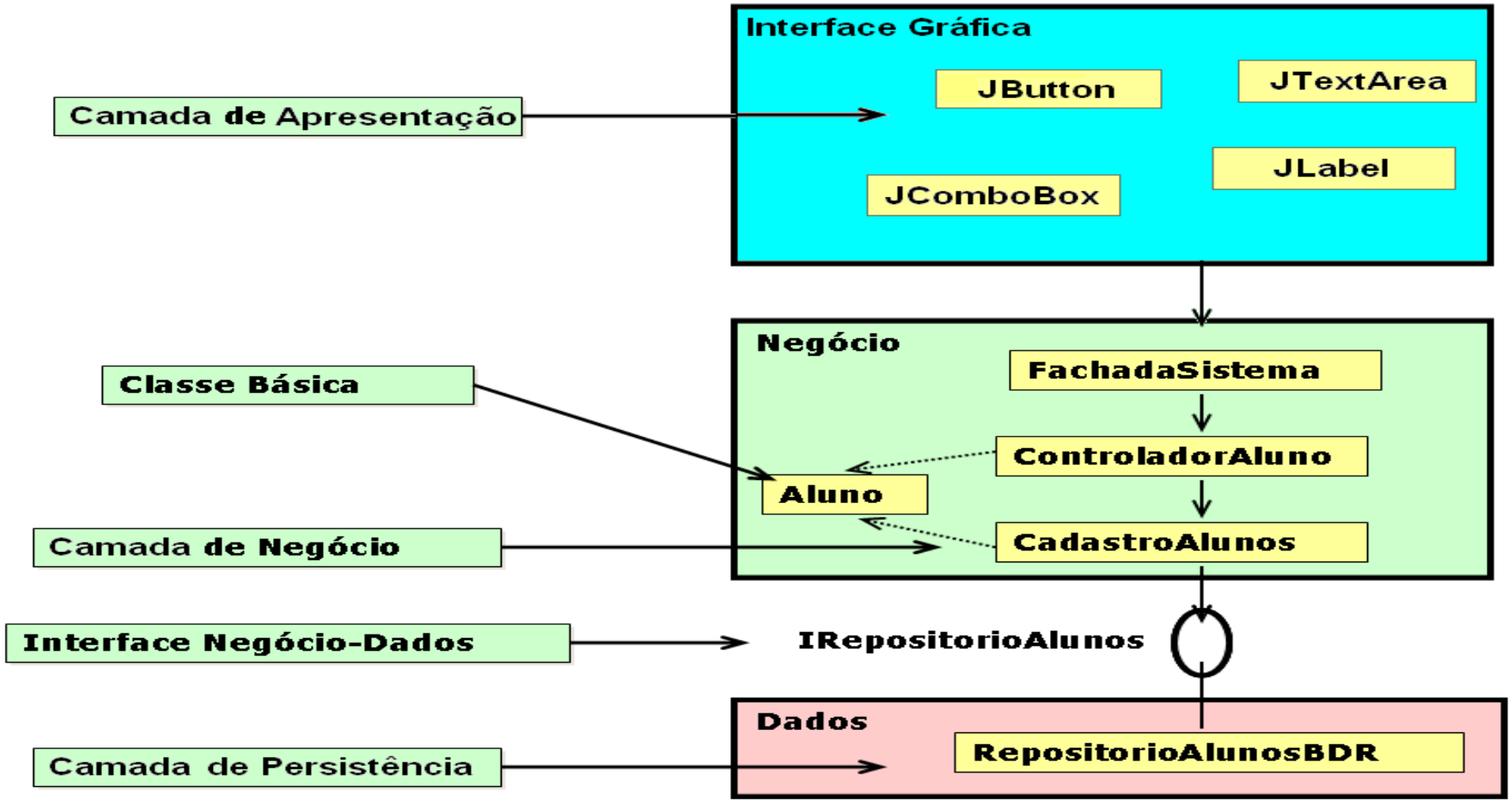
- A arquitetura cliente/servidor em 2 camadas possui como principal problema a dificuldade de manutenção.

- Há uma melhor estruturação e organização da ideia de como montar um projeto de fabricação de software.

Camadas

- Camada de Apresentação
  - Camada que trata a parte de interface gráfica: botões, janelas, frames, áreas de texto, labels...
- Camada de Negócios
  - Camada responsável pela lógica de negócios do sistema. Não é uma camada visível, como a camada de apresentação.
- Camada de Persistência
  - Camada responsável por salvar o estado do objeto em algum lugar (geralmente, numa base de dados).

Exemplos


Interface Gráfica
JButton
JTextArea
JLabel
JComboBox
Camada de Apresentação
Negócio
FachadaSistema
ControladorAluno
Aluno
CadastroAlunos
Classe Básica
Camada de Negócio
Interface Negócio-Dados
IRepositorioAlunos
Dados
RepositorioAlunosBDR
Camada de Persistência

Exemplos

projetoTeste
src
aplicacao
CadastroContas.java
Conta.java
gui
TelaAutenticacao.java
TelaCadastro.java
TelaInicial.java
persistencia
ConexaoBD.java
JRE System Library [JavaSE-1.6]

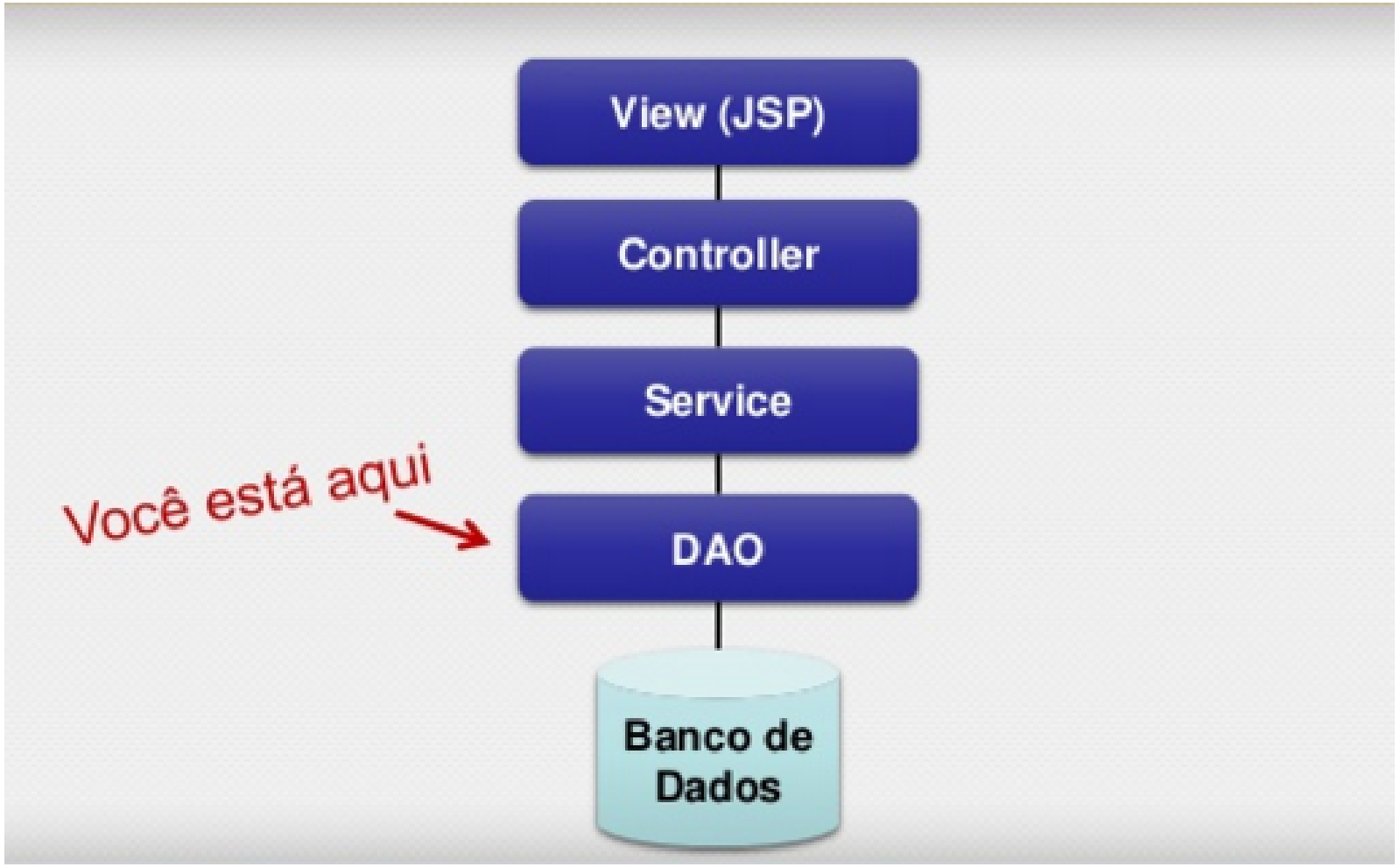
Data Access Object (DAO)

FACULDADE IBRATEC

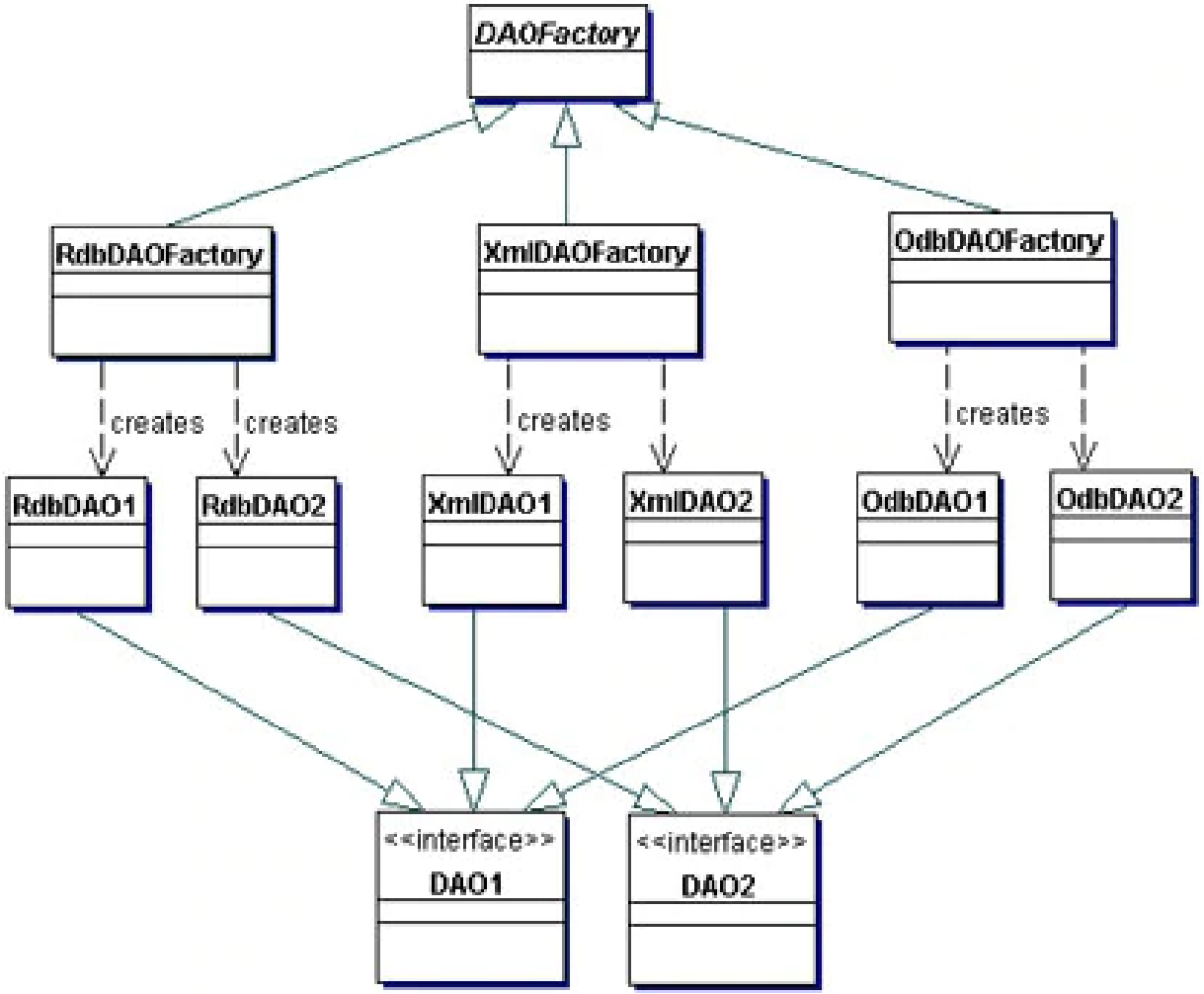
Objetivos

- Os dados podem estar em diversos formatos e fontes
  - ex: XML, JSON, DB, CSV, Texto, Imagem, etc.
- Como padronizar a comunicação com essa variedade de fontes e formatos ?
  - Objetos DAO (Data Access Objects)

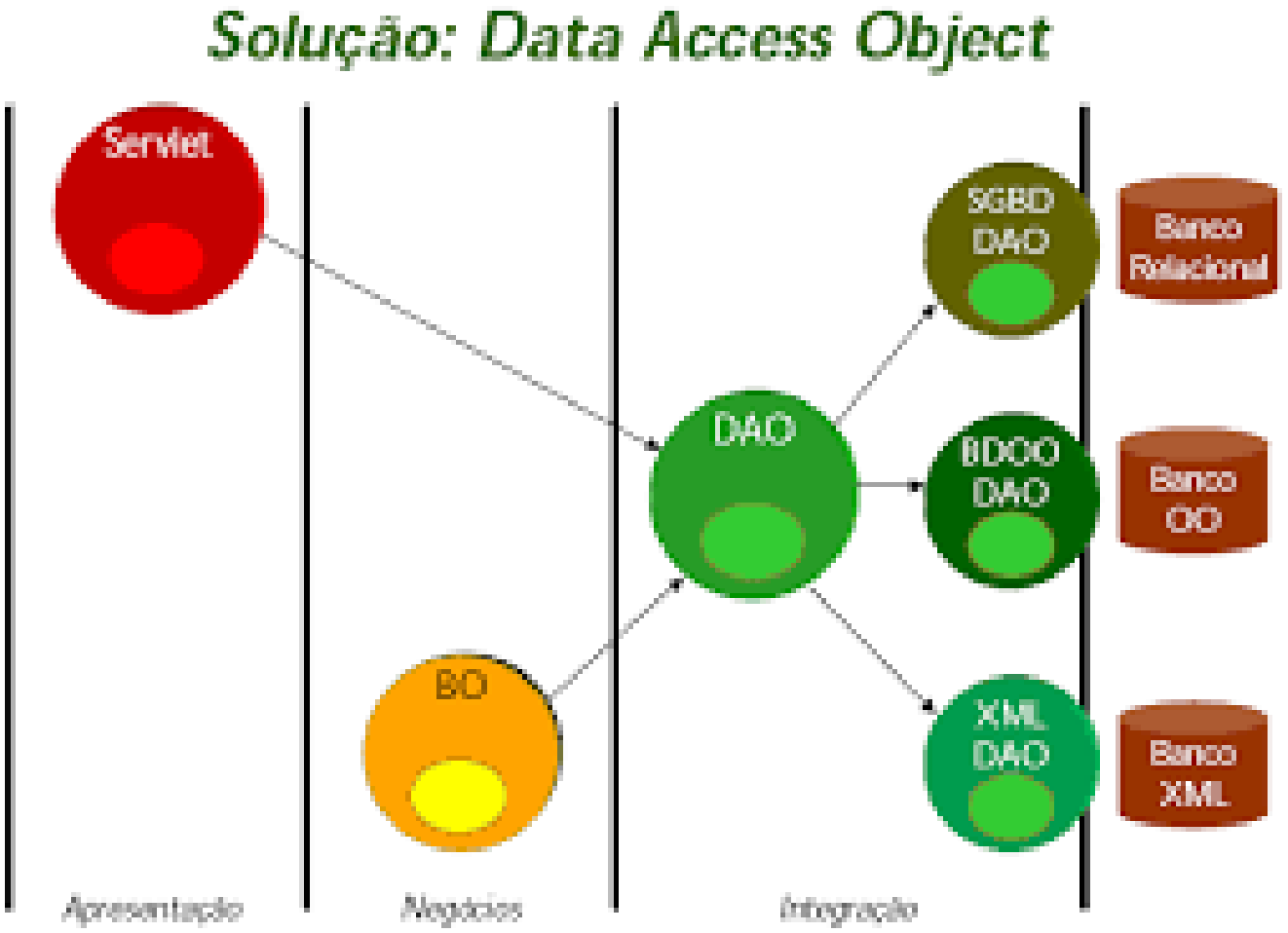
DAO


View (JSP)
Controller
Service
Você está aqui
DAO
Banco de Dados

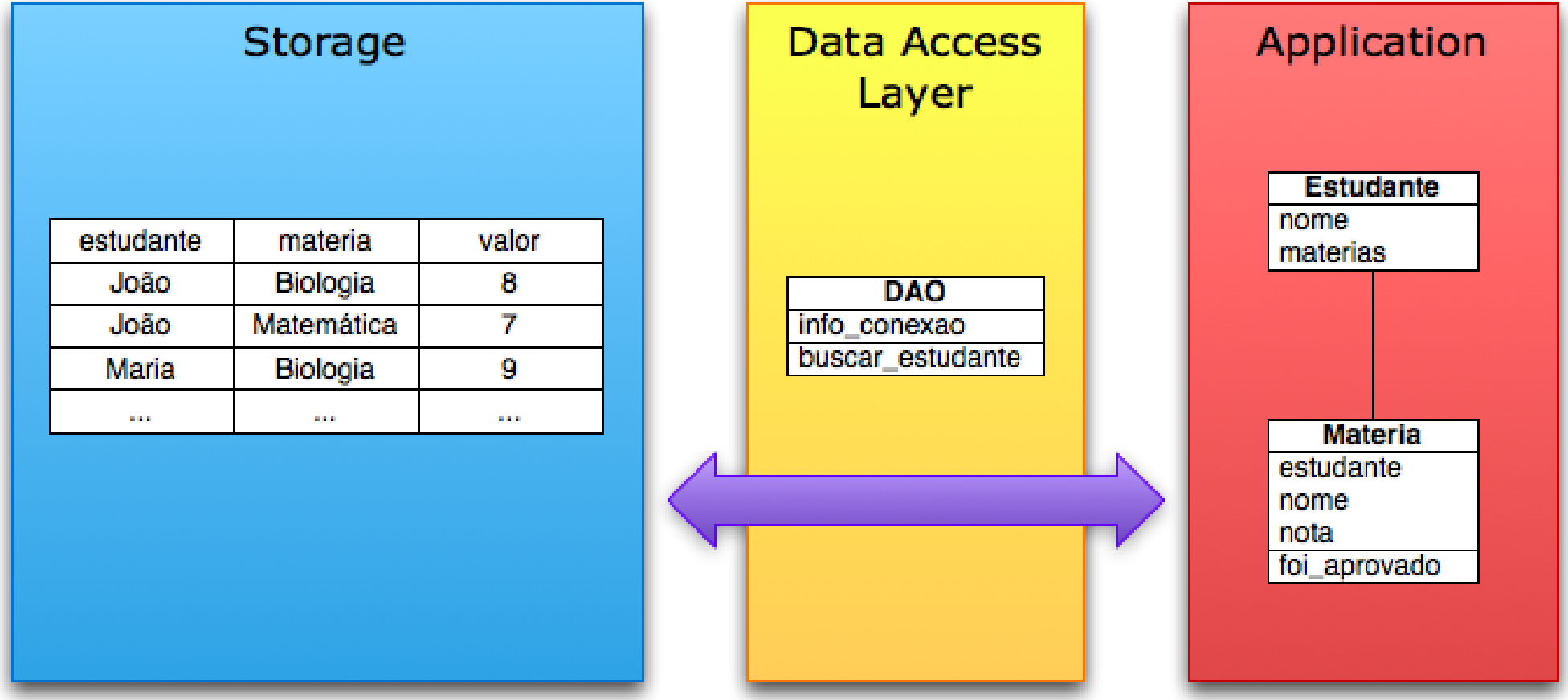
DAO


DAOFactory
RdbDAOFactory
XmlDAOFactory
OdbDAOFactory
creates
creates
creates
creates
RdbDAO1
RdbDAO2
XmlDAO1
XmlDAO2
OdbDAO1
OdbDAO2
<<interface>>
DAO1
<<interface>>
DAO2

DAO


Solução: Data Access Object
Servlet
BO
DAO
SGBD DAO
BDOO DAO
XML DAO
Banco Relacional
Banco OO
Banco XML
Apresentação
Negócios
Integração

DAO


Storage
estudante | materia | valor
João | Biologia | 8
João | Matemática | 7
Maria | Biologia | 9
... | ... | ...
Data Access Layer
DAO
info_conexao
buscar_estudante
Application
Estudante
nome
materias
Materia
estudante
nome
nota
foi_aprovado